

ASSIGNMENT - 7.2

2303A51042

Batch : 29

Task 1 – Runtime Error Due to Invalid Input Type

- A Python program accepts user input and performs arithmetic operations. However, the program throws a runtime error because the input is treated as a string instead of a numeric type.

Example (Buggy Code):

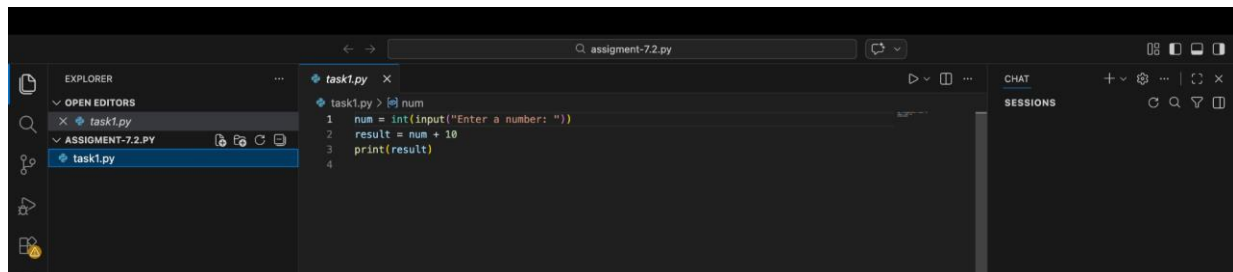
```
num = input("Enter a number: ")
result = num + 10
print(result)
```

- Task:

Use AI tools to identify the cause of the runtime error and modify the program so it executes correctly.

Expected Output -1:

- AI converts the input to the appropriate numeric type and eliminates the runtime error.



Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-7.2.py % /usr/bin/python3 /Users/arepallyramcharan/assignment-6.5.py/assignment-7.2.py/task1.py
Enter a number: 5
15
arepallyramcharan@Arepallys-MacBook-Air assignment-7.2.py %
```

Observation:

The AI tool successfully identified the cause of the runtime error related to incorrect data types and corrected it by converting the input into a numeric type. This task demonstrates how AI-assisted code completion helps in debugging runtime errors, improves program reliability, and reinforces the importance of proper input handling in Python programs.

Task 2 – Incorrect Function Return Value

A function is designed to calculate the square of a number, but it does not return the computed result properly.

Example (Buggy Code):

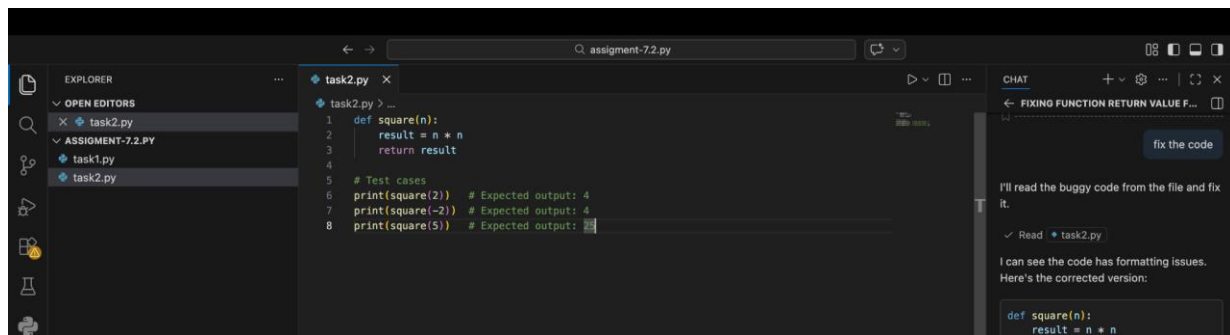
```
def square(n):  
    result = n * n
```

Task:

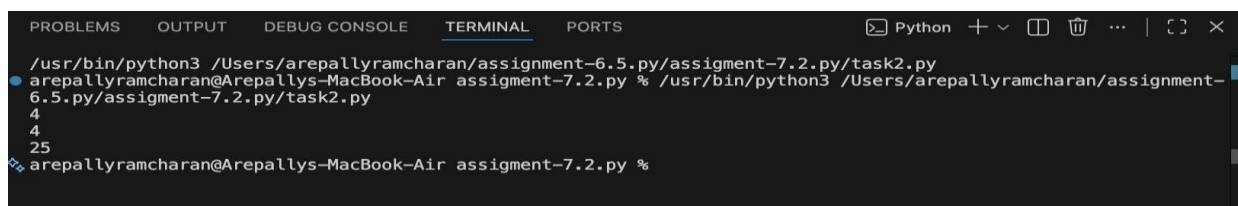
Use AI assistance to analyze the function and ensure the correct value is returned.

Expected Output -2:

AI fixes the missing return statement and the function returns the correct output.



Output:



Observation:

The AI tool successfully identified the logical error caused by the missing return statement and corrected it. This task demonstrates how AI-assisted coding helps detect common function-related mistakes, improves code correctness, and reinforces the importance of returning values from functions in Python programs.

Task 3 – IndexError in List Traversal

A Python program iterates over a list using incorrect index limits, causing an IndexError.

Example (Buggy Code):

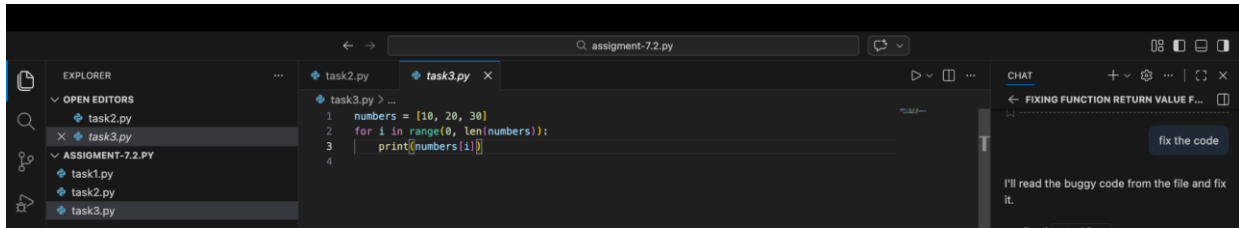
```
numbers = [10, 20, 30]
for i in range(0, len(numbers)+1):
    print(numbers[i])
```

Task:

Use AI to identify the incorrect loop boundary and correct the iteration logic.

Expected Output -3:

AI fixes the loop condition and prevents out-of-range list access.



Output:

```
/usr/bin/python3 /Users/arepallyramcharan/assignment-6.5.py/assignment-7.2.py/task3.py
● arepallyramcharan@Arepallys-MacBook-Air assignment-7.2.py % /usr/bin/python3 /Users/arepallyramcharan/assignment-6.5.py/assignment-7.2.py/task3.py
10
20
30
arepallyramcharan@Arepallys-MacBook-Air assignment-7.2.py %
```

Observation:

The AI tool correctly detected the loop boundary error and fixed it by adjusting the iteration logic. This task highlights how AI-assisted debugging helps prevent common runtime errors, improves loop safety, and encourages best practices in list traversal.

Task 4 – Uninitialized Variable Usage

A program uses a variable in a calculation before assigning it any value.

Example (Buggy Code):

if True:

pass

print(total)

Task:

Use AI tools to detect the uninitialized variable and correct the program.

Expected Output -4:

AI initializes the variable correctly before it is used


```
marks = 85
```

```
if marks >= 90:
```

```
    grade = "A"
```

```
elif marks >= 80:
```

```
    grade = "C"
```

```
else:
```

```
    grade = "B"
```

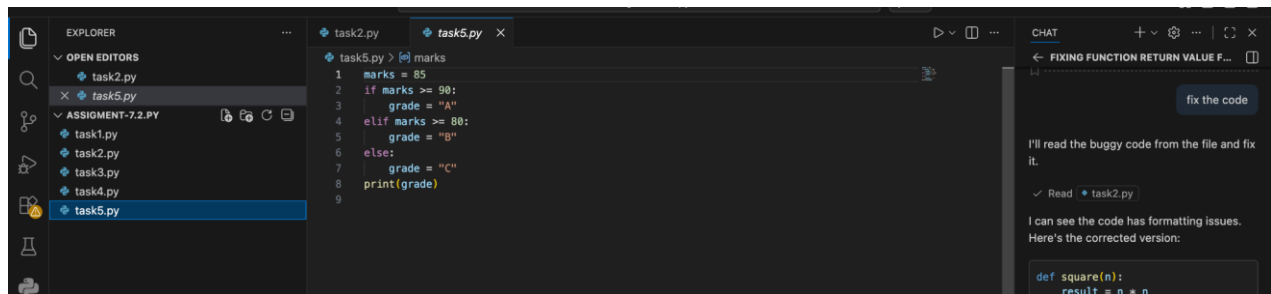
```
print(grade)
```

Task:

Use AI to analyze the grading conditions and correct the logical flow.

Expected Output -5:

AI corrects the conditional logic so grades are assigned accurately.



Output:

